Scaling WebSockets is fundamentally different from scaling stateless HTTP APIs because **connections are long-lived, stateful, and resource-intensive**.

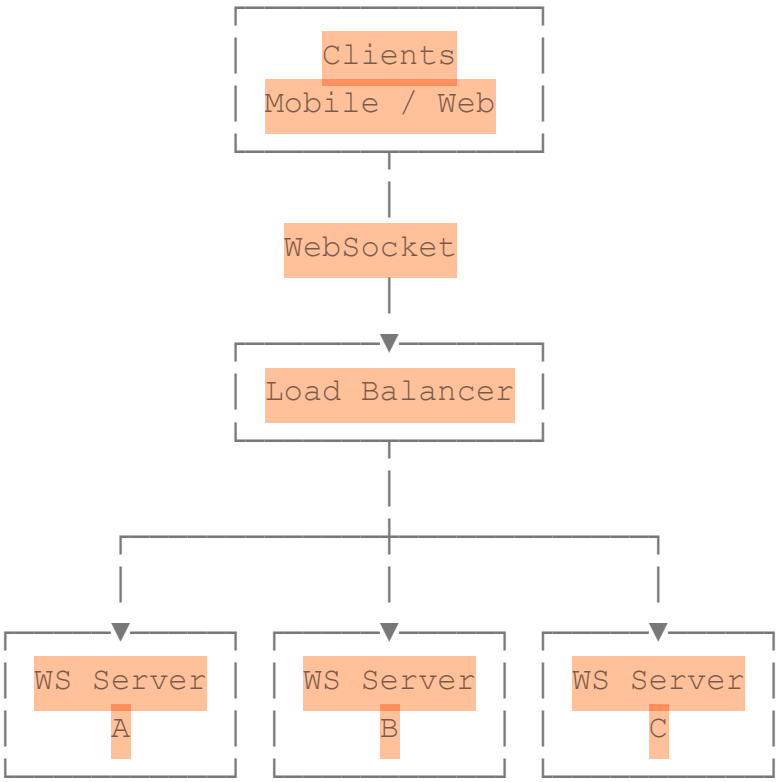
For an HLD/system-design interview, the core question is:

> **How do we support millions of concurrent, long-lived connections while maintaining low latency, connection reliability, message delivery semantics, and horizontal scalability?**

---

1. Start with the basic architecture

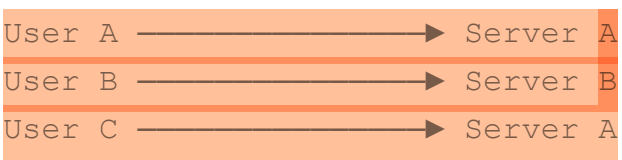
A simple WebSocket system looks like:



The first important property is:

Each WebSocket connection remains attached to one WebSocket server.

For example:



Unlike ordinary HTTP requests, subsequent messages from User A generally return to Server A through that existing connection.

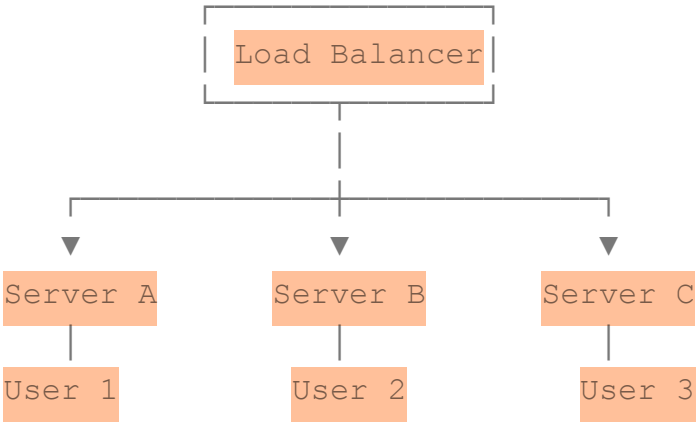
That creates the central scaling problem:

> **What happens when Server B needs to send a message to User A, who is connected to Server A?**

---

2. The key scaling problem: connection locality

Suppose we have:



Now User 2 sends:

```
"Send message to User 1"
```

Server B knows the message needs to go to User 1.

But User 1 is connected to:

```
Server A
```

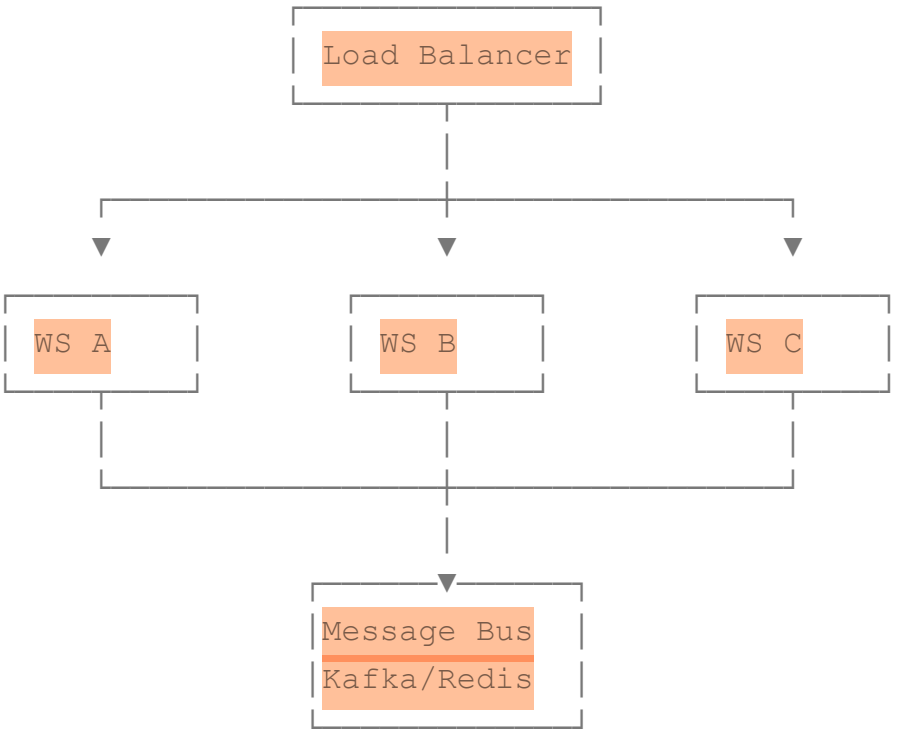
Server B cannot simply write to User 1's socket.

We therefore need **cross-node message routing**.

---

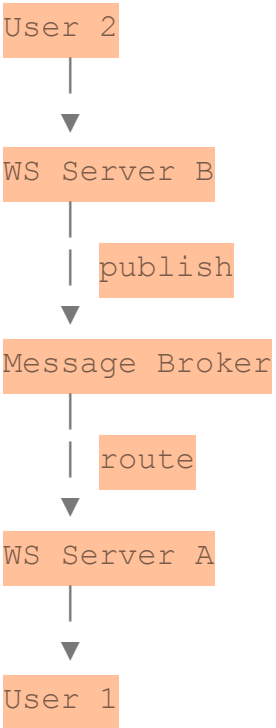
3. Introduce a message broker

A common architecture is:



Each WebSocket server can communicate through the broker.

For example:



This is the fundamental pattern for horizontally scaling WebSockets.

---

4. Connection registry

We also need to know:

> **Which server owns a particular connection?**

Maintain a distributed registry:

```
userId → WebSocket server
```

For example:

```
Redis

user:1001 → ws-server-17
user:1002 → ws-server-03
user:1003 → ws-server-17
```

More realistically:

```
connectionId → serverId
```

because one user can have multiple connections:

```
user:1001
   │
   ├── connection-A → server-17
   ├── connection-B → server-22
   └── connection-C → server-22
```

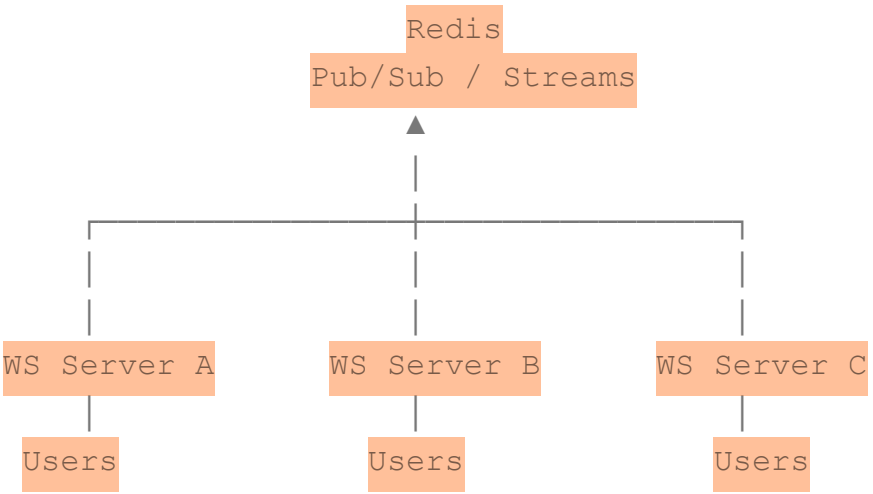
This matters because a user might simultaneously have:

- laptop
- mobile
- tablet
- multiple browser tabs

---

5. Redis Pub/Sub architecture

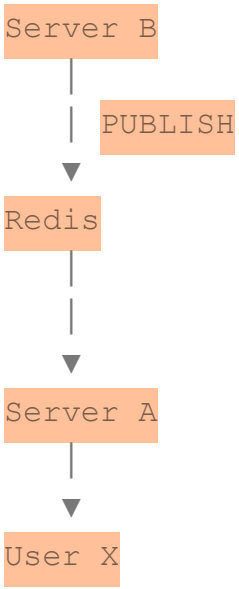
For relatively simple real-time applications:



Suppose:

```
User X → Server A
User Y → Server B
```

User Y sends a message to X.



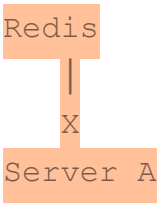
This solves the cross-server communication problem.

---

6. But Redis Pub/Sub has an important limitation

Redis Pub/Sub is generally **ephemeral**.

If Server A disconnects from Redis:

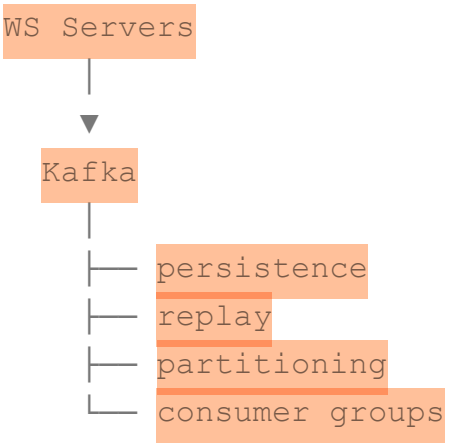


messages published during that period can be lost.

Pub/Sub doesn't give you durable replay semantics.

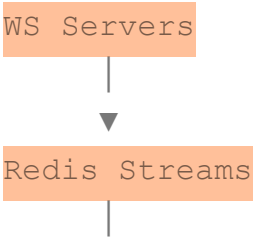
Therefore, if message durability matters, consider:

Kafka



Or:

Redis Streams

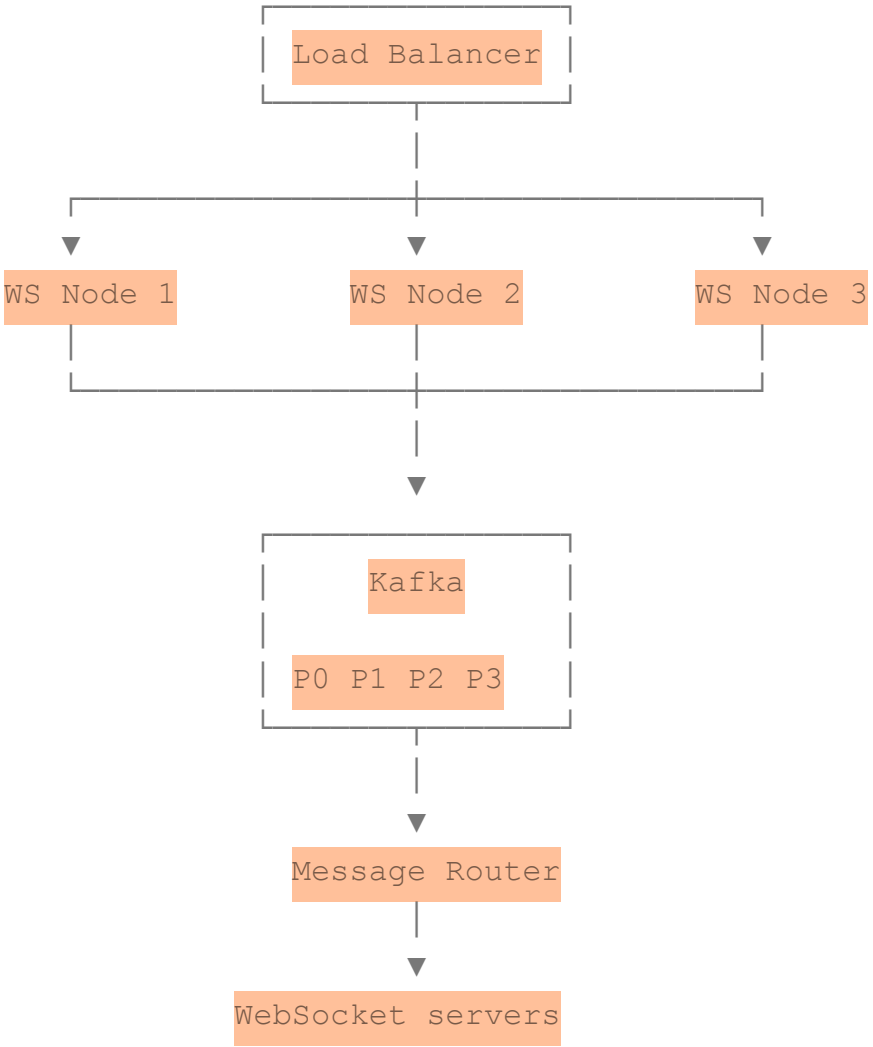


```
├── persistence
├── consumer groups
└── acknowledgements
```

---

7. Kafka-based architecture

For a high-scale system:



Kafka provides:

- durable messages
- ordering within partitions
- replay
- horizontal scaling
- backpressure
- consumer groups

But there is an architectural question:

> How do we efficiently route a message to the server holding the target connection?

---

8. Partition by user ID

A common approach is:

```
partition = hash(userId) % N
```

For example:

```
user 1001 → partition 3
user 1002 → partition 7
user 1003 → partition 3
```

This gives predictable routing.

However, don't confuse:

Kafka partition ownership

with

WebSocket connection ownership.

They are related routing mechanisms but solve different problems.

---

9. Sticky sessions

Another common technique is **sticky sessions**.

The load balancer tries to keep a client attached to the same WebSocket server.

```
Client A ──────────► Server A

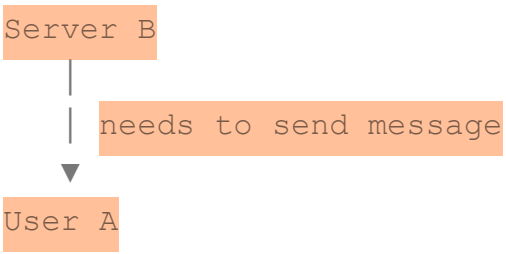
Client B ──────────► Server B

Client C ──────────► Server C
```

Sticky sessions are useful, but they are **not sufficient for distributed WebSocket architecture**.

Why?

Because:



User A still lives on Server A.

You still need inter-node communication.

Also, sticky sessions create problems during:

- server failure
- scaling
- deployments
- load imbalance

Therefore:

> **Sticky sessions can help connection affinity, but don't use them as the primary distributed messaging strategy.**

---

10. WebSocket load balancer

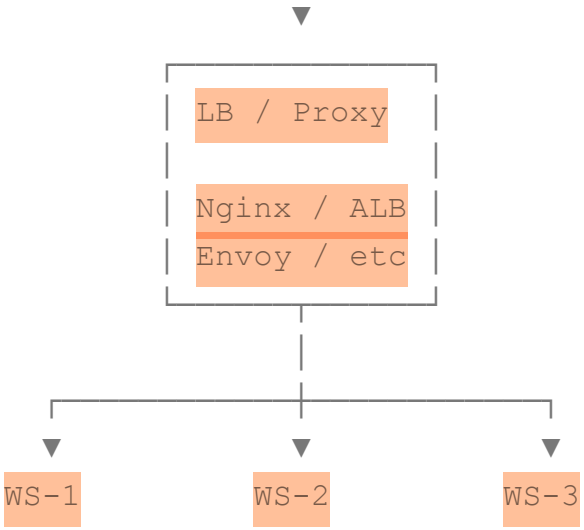
The load balancer must support WebSocket upgrade:

```
GET /ws HTTP/1.1
Upgrade: websocket
Connection: Upgrade
```

Then the connection becomes a long-lived TCP connection.

Architecture:





The load balancer needs to account for:

- WebSocket upgrade
- connection timeout
- idle timeout
- health checks
- connection draining
- TLS termination
- maximum connections
- connection distribution

---

11. Heartbeats

A major issue with long-lived connections is detecting dead connections.

Consider:

```
Client
  |
  | TCP connection
  |
  X Network disappears
  |
Server doesn't immediately know
```

Use:

```
PING
  ↓
PONG
```

For example:

```
Server ——— PING ———▶ Client
Server ◀——— PONG ——— Client
```

If the client doesn't respond after a threshold:

```
connection = dead
```

Then:

```
close connection
remove registry entry
release resources
```

---

12. Connection registry must have TTL

Suppose:

```
user-123 → ws-17
```

but ws-17 crashes.

The registry must eventually remove the stale entry.

Therefore:

```
Redis

user-123 → ws-17
           TTL: 60 sec
```

The WebSocket server periodically refreshes its registration.

This is essentially a **lease**.

If the server dies:

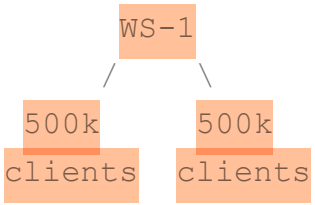
```
lease expires
      ↓
mapping disappears
```

This prevents routing messages to dead nodes.

---

13. Handling server failures

Suppose:



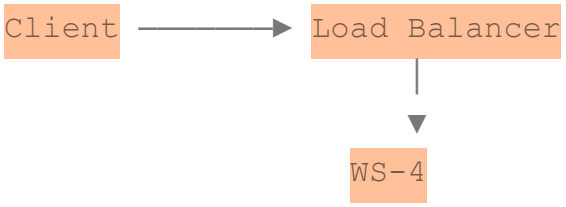
WS-1 crashes.

Its 500k connections disappear.

Clients need automatic reconnection:

```
Client
  │
  │ connection lost
  ▼
Exponential backoff
  │
  ├── 1 sec
  ├── 2 sec
  ├── 4 sec
  ├── 8 sec
  └── ...
```

Then:



Important:

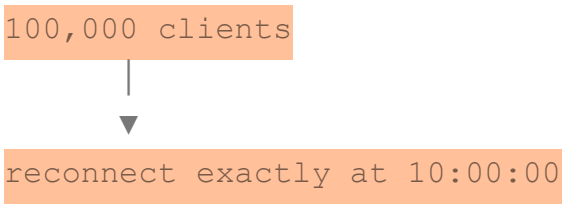
> The server should not immediately reconnect thousands/millions of clients simultaneously.

Otherwise you create a **reconnection storm**.

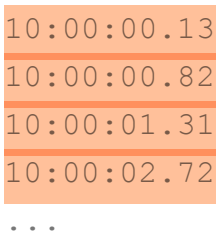
---

14. Jitter

Instead of:



use:



Combine:

exponential backoff + random jitter

to spread reconnection load.

---

15. Connection capacity

For WebSockets, scaling isn't only CPU.

A useful capacity model is:

```
Total connections
        =
nodes × connections/node
```

Suppose:

```
1 server = 100,000 connections

10 servers = 1,000,000 connections
```

But this number depends heavily on:

- message rate
- message size
- TLS
- memory
- kernel configuration
- network bandwidth
- heartbeat frequency
- application logic
- serialization
- runtime

100k mostly-idle connections are very different from:

```
100k connections
×
10 messages/sec
```

---

16. Memory becomes important

Every connection consumes resources.

Conceptually:

```
Connection
    |
```

```
    ├── socket
    ├── TCP state
    ├── buffers
    ├── WebSocket state
    ├── authentication context
    ├── subscriptions
    └── application metadata
```

Therefore:

```
Connections × memory/connection
```

becomes a major capacity constraint.

For example:

```
1,000,000 connections
×
20 KB
≈
20 GB
```

And that's before considering application/runtime overhead.

Therefore, don't put excessive state inside each WebSocket process.

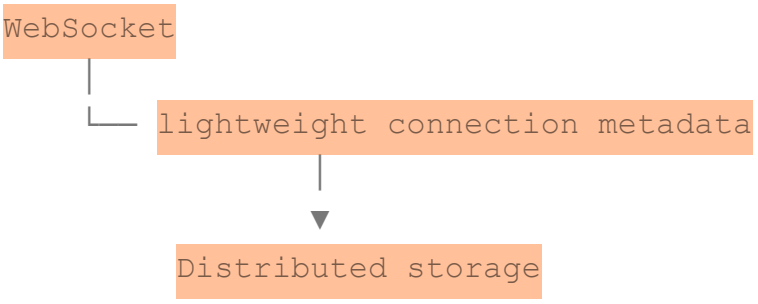
---

17. Avoid large per-connection state

Bad:

```
WebSocket connection
        │
        ├── entire user profile
        ├── entire chat history
        ├── large caches
        └── large subscription data
```

Prefer:



For example:

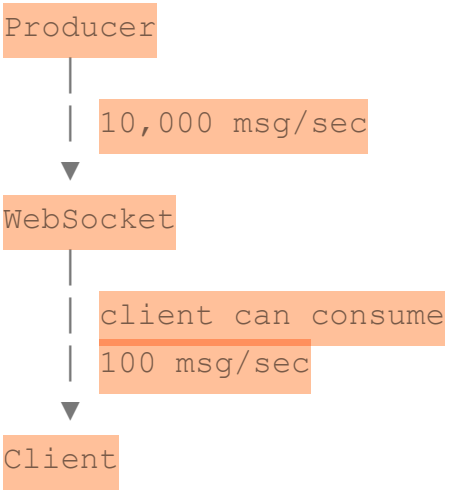
```
connectionId
userId
serverId
subscriptions
lastHeartbeat
```

---

18. Backpressure

This is one of the most important advanced WebSocket concepts.

Suppose:



The outgoing buffer grows indefinitely.

Eventually:

```
memory → exhausted
```

You need backpressure.

Possible strategies:

A. Bounded queue

```
max queue = 1,000 messages
```

When full:

```
drop
reject
disconnect
or apply policy
```

B. Coalescing

Instead of:

```
price = 100
price = 101
price = 102
price = 103
...
```

send:

```
latest price = 103
```

Useful for:

- stock prices
- telemetry
- GPS
- dashboards

C. Sampling

Send:

```
10 updates/sec
```

instead of:

```
1,000 updates/sec
```

---

19. Message semantics

You should explicitly define:

At-most-once

```
send → maybe delivered
```

No retries.

At-least-once

```
send
 ↓
retry if necessary
```

Duplicates are possible.

Exactly-once

Much harder across distributed systems.

For many WebSocket systems, the practical model is:

> **At-least-once delivery + idempotency/deduplication.**

Use:

```
messageId
```

For example:

```
{
  "messageId": "msg-12345",
  "type": "CHAT_MESSAGE",
  "senderId": "u1",
  "recipientId": "u2",
  "timestamp": 1789560000
}
```

Client/server can deduplicate using `messageId`.

---

20. Ordering

WebSocket itself preserves ordering **within a single connection**.

For example:

```
A
B
C
```

will arrive in that order on that connection.

But distributed systems complicate this.

Consider:

```
Server A → message A
Server B → message B
```

These may reach downstream systems differently.

If strict ordering is required:

```
partition by conversationId
```

For chat:

```
Kafka key = conversationId
```

Then all messages for a conversation go to the same partition:

```
conversation-123
        |
        ▼
Kafka Partition 7

M1
M2
M3
M4
```

Kafka maintains ordering within that partition.

---

21. Multi-device users

Suppose:

```
User 123

Laptop ⟶ WS-1
Phone  ⟶ WS-7
Tablet ⟶ WS-3
```

A message to User 123 may need to be delivered to:

```
WS-1
WS-7
WS-3
```

Therefore the registry should support:

```
userId → Set<connectionId>
```

rather than:

```
userId → one server
```

Example:

```
user-123

connection-A → WS-1
connection-B → WS-7
connection-C → WS-3
```

---

22. Presence

Chat applications often need:

```
online
offline
last seen
typing
```

Don't rely solely on WebSocket connection state.

Maintain distributed presence:

```
Redis

presence:user123
    status = ONLINE
    server = ws-17
    lastHeartbeat = ...
    TTL = 30 sec
```

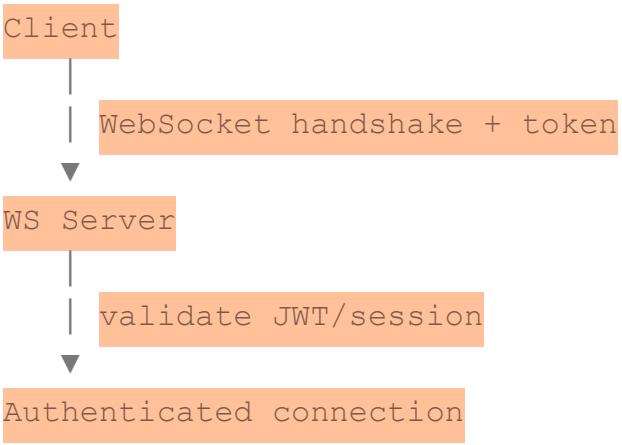
TTL expiration naturally helps detect stale presence.

---

23. Authentication

Authenticate during the WebSocket handshake.

Typical flow:

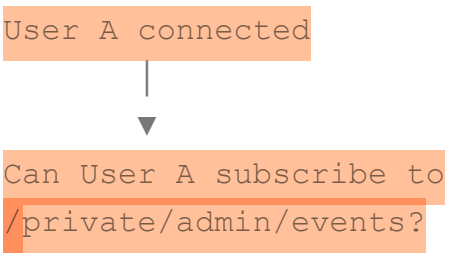


But remember:

> Authentication ≠ authorization.

A connected user still needs authorization for each operation.

For example:



That must be checked.

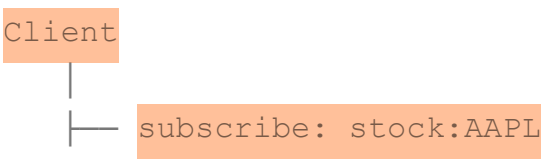
---

24. Subscription architecture

For systems like:

- stock feeds
- notifications
- live dashboards
- sports scores
- IoT telemetry

clients subscribe to topics:



```
├── subscribe: stock:MSFT
└── subscribe: market:NASDAQ
```

The WebSocket node maintains:

```
topic → local connections
```

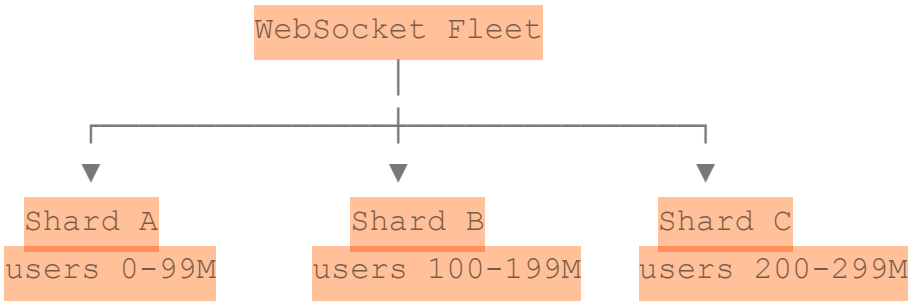
while the distributed messaging layer handles cross-node propagation.

---

25. Sharding

At very large scale, partition connections.

For example:



Possible partition keys:

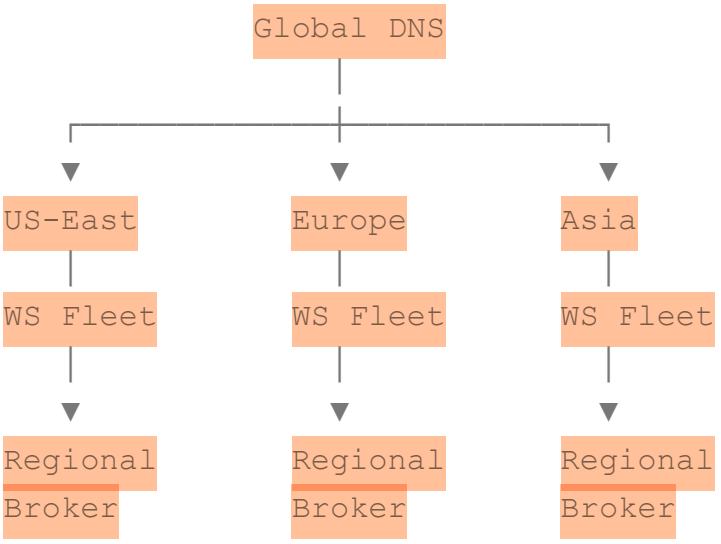
```
userId
tenantId
region
conversationId
```

Choice depends on access patterns.

---

26. Multi-region architecture

At global scale:



Prefer users connecting to a nearby region.

For example:

```
India → Asia region
Germany → Europe region
USA → US region
```

This reduces:

- connection latency
- heartbeat latency
- bandwidth costs
- cross-region traffic

---

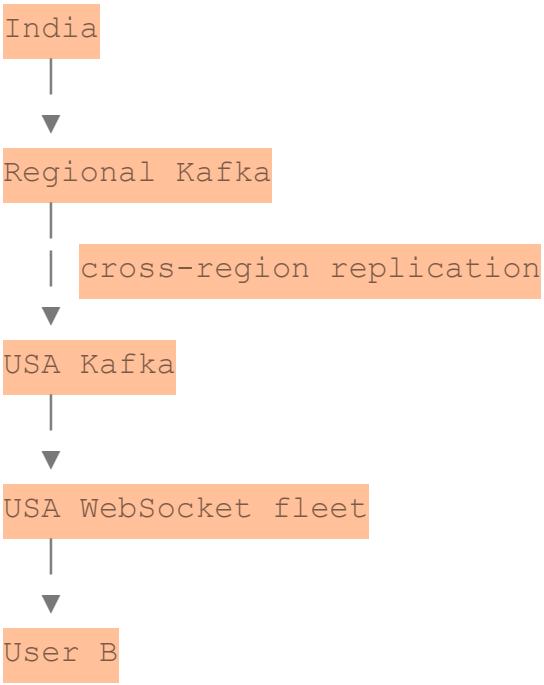
27. Global messaging is harder

Suppose:

```
User A → India
User B → USA
```

A sends a message to B.

You need:



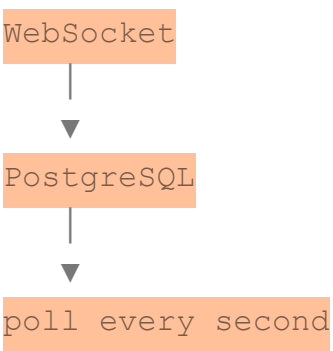
You then need to reason about:

- ordering
- replication latency
- duplicate delivery
- regional failures
- data sovereignty
- failover
- consistency

---

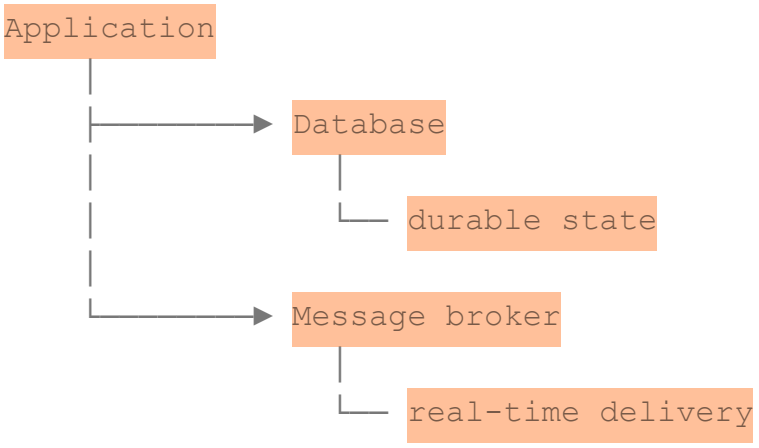
28. Database is usually NOT the messaging layer

Avoid:



for high-scale real-time delivery.

Instead:



Database:

> source of truth

Broker:

> event/message distribution

WebSocket:

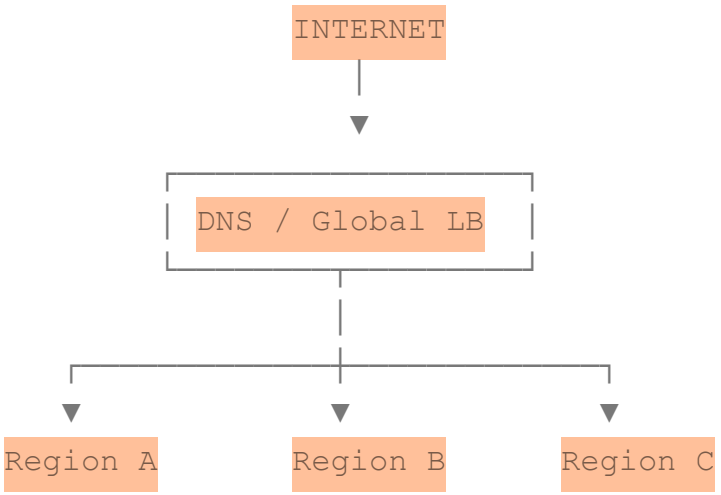
> client delivery channel

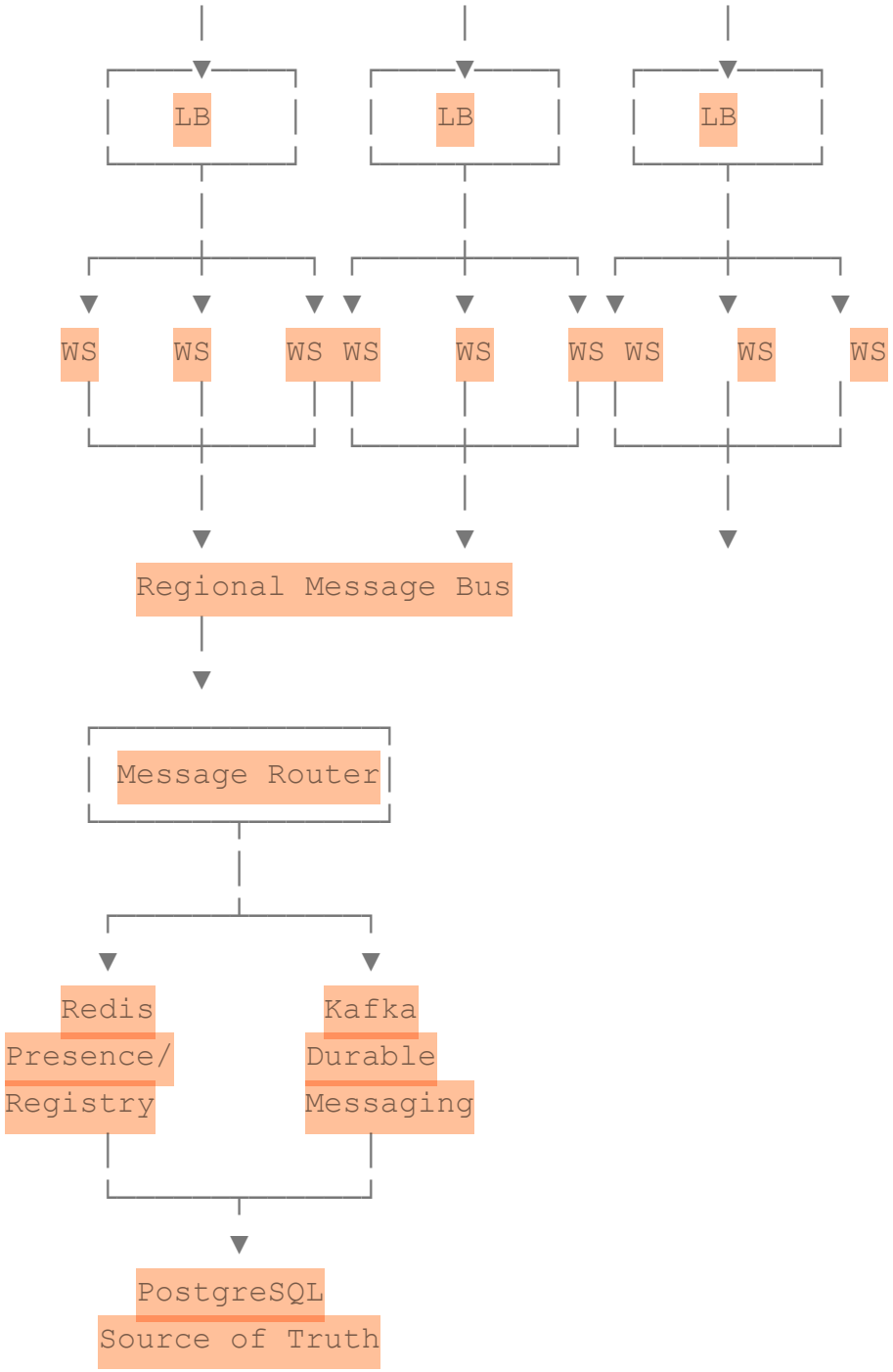
This separation is extremely useful in system design.

---

29. Recommended production architecture

For a serious large-scale system, I'd typically draw:





---

30. How to scale each layer

Layer	Scaling strategy
DNS / Global LB	Multi-region routing
Load Balancer	Horizontal scaling / managed LB
WebSocket servers	Horizontal scaling
Connection registry	Redis cluster/sharding

Layer	Scaling strategy
Message broker	Kafka partitions / Redis Cluster
Database	Read replicas + sharding where necessary
Presence	Redis + TTL
Message history	PostgreSQL / NoSQL/object storage depending on workload
Observability	Metrics/log aggregation/tracing
Client reconnection	Exponential backoff + jitter

---

31. Autoscaling WebSocket servers is different

HTTP autoscaling commonly looks at:

```
CPU
RPS
latency
```

For WebSockets, these aren't sufficient.

Monitor:

```
active_connections
connections_per_node
connection_rate
disconnection_rate
messages_in/sec
messages_out/sec
bytes_in/sec
bytes_out/sec
event_loop_lag
memory
GC pauses
```

```
outbound_queue_depth
heartbeat_failures
```

A particularly important metric:

```
connections per node
```

For example:

```
Target = 80,000 connections/node
```

```
Node 1 = 82k
Node 2 = 78k
Node 3 = 81k
```

Trigger scaling before nodes become saturated.

---

32. The deployment problem

This is an important interview question.

Suppose:

```
WS-1
  └── 100,000 active connections
```

You deploy version 2.

You cannot simply kill WS-1.

Otherwise:

```
100,000 clients
      ↓
reconnect
      ↓
massive connection storm
```

Instead use **connection draining**.

```
WS-1
  │
  ├── stop accepting new connections
  │
  ├── existing connections continue
  │
  └── gradually close/migrate
```

Clients reconnect to new nodes.

This gives:

```
Rolling deployment
+
connection draining
+
backoff/jitter
```

---

33. Observability

At scale, track:

Connection metrics

```
active_connections
new_connections/sec
disconnects/sec
connection_duration
```

Network

```
bytes_in
bytes_out
```

Messaging

```
messages/sec
broker_lag
queue_depth
delivery_latency
```

Reliability

```
connection_failures
heartbeat_failures
reconnect_rate
dropped_messages
duplicate_messages
```

Node health

```
CPU
memory
GC
event-loop lag
file descriptors
TCP sockets
```

---

34. Important bottleneck: file descriptors

Each TCP connection consumes a file descriptor.

If:

```
ulimit = 65,535
```

you cannot expect a process to comfortably manage hundreds of thousands of sockets without appropriate OS/runtime configuration.

For Linux-based systems, investigate:

```
ulimit
epoll
```

```
TCP backlog
socket buffers
conntrack
```

and application/runtime limits.

This is a very good low-level detail to mention in senior system-design interviews.

---

35. WebSocket vs SSE

An interviewer may ask:

> Why WebSocket instead of Server-Sent Events?

WebSocket

Bidirectional:

```
Client ◀──────────▶ Server
```

Good for:

- chat
- multiplayer gaming
- collaborative editing
- trading
- interactive applications

SSE

Server → client:

```
Client ◀────────── Server
```

Good for:

- notifications
- dashboards
- live feeds
- streaming updates

If clients don't need bidirectional communication, SSE can simplify architecture.

---

36. WebSocket vs polling

Polling:

```
Client → GET
Client → GET
Client → GET
Client → GET
```

WebSocket:

```
Client ═══════════════ Server
          persistent
```

WebSocket reduces unnecessary request overhead and provides much lower-latency server push.

But the trade-off is:

> **You now have to manage millions of stateful connections.**

---

37. The interview answer I'd memorize

If the interviewer says:

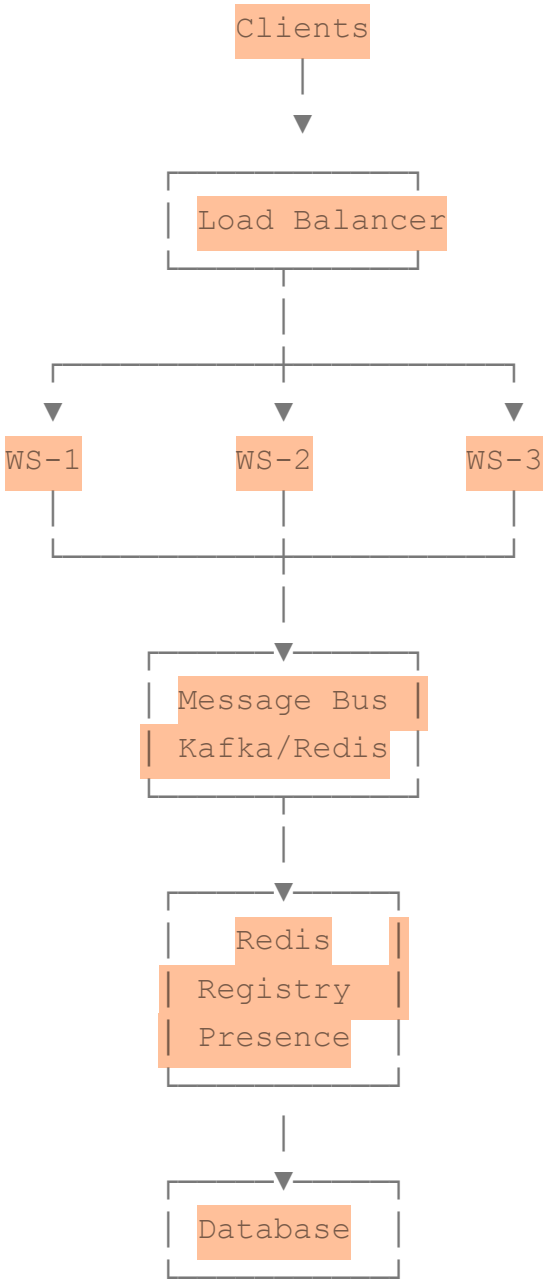
> **"Design a scalable WebSocket system."**

Start with this:

> "WebSocket connections are long-lived and stateful, so I would horizontally scale a fleet of WebSocket gateway servers behind a load balancer. Each connection is owned by one gateway. To communicate across gateways, I'd introduce a distributed message bus such as Kafka or Redis Streams, and maintain a distributed connection/presence registry, typically Redis, mapping users or connection IDs to gateway nodes. I'd use heartbeats and TTL-based leases for connection health, bounded outbound queues for backpressure, and client-side exponential backoff with jitter for reconnects. For high availability I'd use multiple gateway instances across availability zones and, at global scale, regional WebSocket fleets with regional brokers. Durable

> application state remains in the database, while the broker handles real-time event distribution."

Then draw:



---

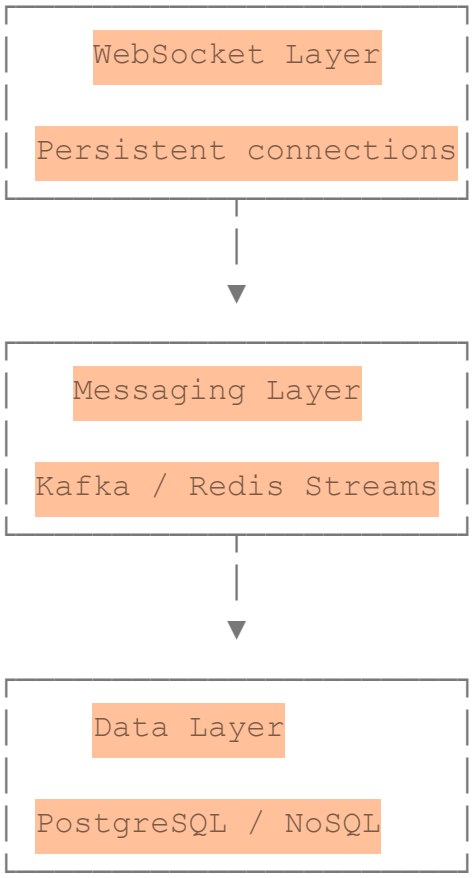
38. The 10 things an interviewer expects you to discuss

For a **MAANG+ senior/staff-level WebSocket HLD**, make sure you cover these:

1. **Horizontal scaling of WebSocket gateways**
2. **Connection ownership/locality**

3. **Cross-node message routing**
4. **Connection registry / presence**
5. **Redis vs Kafka vs Redis Streams**
6. **Heartbeats and dead connection detection**
7. **Backpressure and slow consumers**
8. **Delivery semantics + idempotency**
9. **Reconnect storms + connection draining**
10. **Multi-region architecture and failure handling**

The most important conceptual separation to internalize is:




```
WebSocket = connection delivery
Broker    = event distribution
Database  = durable state
```

That separation is the **core design principle** behind scalable WebSocket systems.